DOCUMENTO TÉCNICO · PROYECTO FINAL

AtteliA

Asesora inteligente de moda basada en RAG, LangChain y LangGraph

Modalidad: proyecto académico de aplicación web inteligente

Integrante(s): [COMPLETAR ANTES DE LA ENTREGA]

Asignatura / profesor: [COMPLETAR]

Fecha: 18 de octubre de 2026

Repositorio GitHub: [URL PENDIENTE DE PUBLICACIÓN]

Frontend y backend públicos: [URL PENDIENTE DE DESPLIEGUE]

ESTADO DE VERIFICACIÓN

Esta memoria describe el código disponible y las modificaciones incorporadas. Se probaron tres escenarios HTTP automáticos; las pruebas reales con Ollama, el despliegue en nube, la carga a GitHub y las capturas visuales requieren acciones y verificaciones adicionales en la máquina del equipo. No se presentan resultados no observados como realizados.

Índice de contenidos

· 1. Nombre del proyecto e integrantes

· 2. Tema seleccionado y descripción del problema

· 3. Arquitectura general y diagrama

· 4. Tecnologías utilizadas

· 5. Descripción del backend

· 6. Descripción del frontend

· 7. Descripción del agente y flujo LangGraph

· 8. Modelo local y base vectorial

· 9. Carga y procesamiento documental

· 10. Estrategias de control de alucinaciones

· 11. Seguridad y autenticación

· 12. Despliegue

· 13. Evidencias de funcionamiento

· 14. Problemas encontrados y conclusiones

· Anexos: trazabilidad, pruebas, configuración y sustentación

Documento elaborado con base en los 14 apartados exigidos en la sección 5.2 del enunciado «Desarrollo de un agente inteligente con LangChain, LangGraph y RAG». Se conserva el alcance propio del sistema AtteliA; los componentes no comprobados se identifican de forma expresa.

1. Nombre del proyecto e integrantes

Nombre: AtteliA — Asesora Inteligente de Moda y Estilo. Nombre de trabajo/repositorio: armario-ia. Integrantes: [COMPLETAR]. Curso: [COMPLETAR].

El sistema ofrece una interfaz conversacional para responder dudas sobre selección de prendas, calzado, combinaciones cromáticas, accesorios y vestimenta por ocasión a partir de documentos indexados.

2. Tema seleccionado y descripción del problema

2.1 Situación problema

La consulta de recomendaciones de moda mediante asistentes generativos sin acceso controlado a fuentes puede producir sugerencias sin trazabilidad. En un trabajo académico orientado a RAG, este comportamiento impide demostrar qué fragmento documental sustentó la respuesta. Adicionalmente, un corpus estático limita la actualización y la personalización de conocimientos.

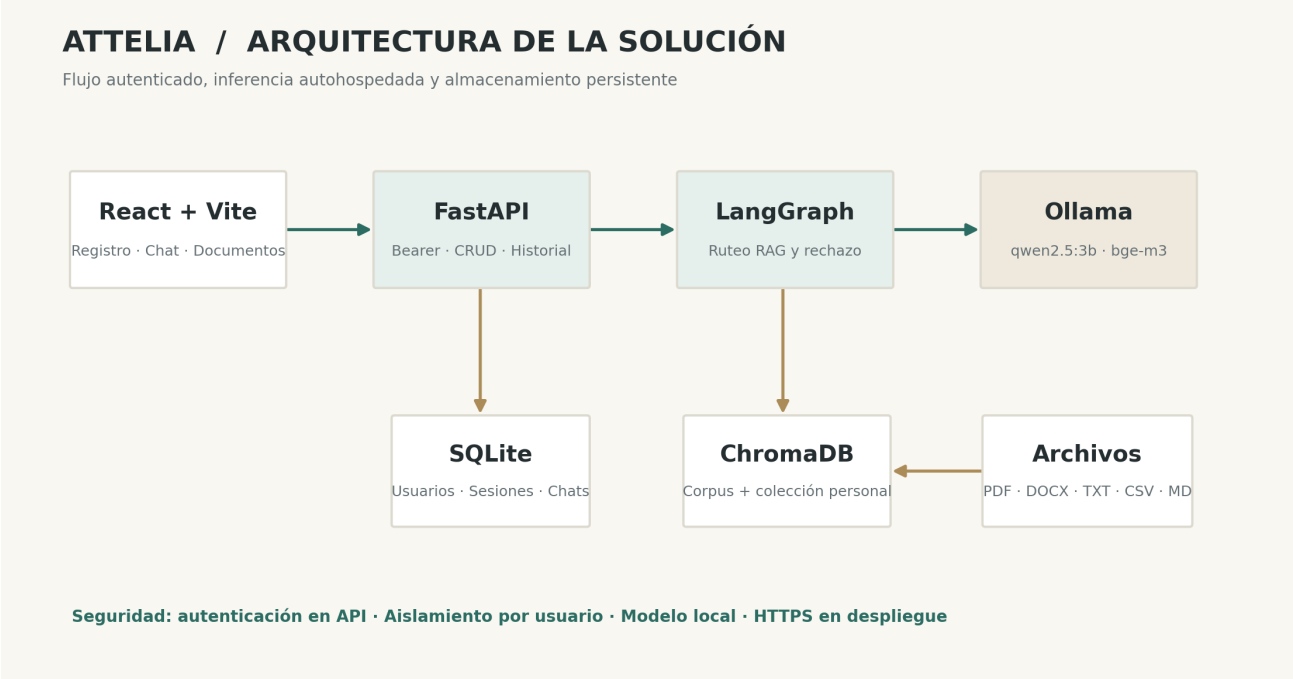
2.2 Propósito y alcance

- Ofrecer autenticación, acceso al chatbot e historial persistente por usuario.
- Responder únicamente si la recuperación semántica devuelve contexto considerado suficiente.
- Permitir subir, listar y eliminar documentos propios en PDF, DOCX, TXT, Markdown o CSV.
- Exponer fuentes consultadas y rechazar preguntas ajenas a la base documental.

2.3 Límites

AtteliA es una herramienta informativa sobre moda, no un probador virtual, un clasificador de imágenes ni una plataforma comercial. El repositorio contiene código experimental de Polyvore, pero el flujo RAG principal de esta entrega usa el corpus textual de seis volúmenes y los archivos agregados por cada usuario.

3. Arquitectura general y diagrama



Elaboración propia a partir del código fuente de AtteliA.

El navegador realiza peticiones HTTP a una API FastAPI protegida por tokens Bearer. FastAPI mantiene usuarios, sesiones, historial y metadatos documentales en SQLite. Para responder, invoca un grafo compilado de LangGraph que recupera pasajes desde Chroma y delega en Ollama la clasificación,

generación y revisión. Los embeddings bge-m3 y la inferencia qwen2.5:3b se ejecutan en el servidor donde corre Ollama, no en servicios comerciales de IA.

3.1 Fronteras de confianza y datos

Las colecciones personales de Chroma tienen nombres derivados del identificador interno del usuario; FastAPI decide el identificador a partir de la sesión autorizada, nunca de un campo enviado por el cliente. Los seis volúmenes son una colección común consultable por cualquier usuario autenticado. La API no acepta rutas arbitrarias de almacenamiento enviadas por el navegador.

4. Tecnologías utilizadas

Componente	Tecnología	Justificación
Interfaz	React 18 + Vite + CSS + Lucide	SPA ligera y componentes reutilizables; mantiene estética AtteliA.
Servidor	Python + FastAPI	Validación de tipos, endpoints REST, subida multipart y documentación interna.
Agente	LangChain + LangGraph	Abstracción de mensajes/modelos y control explícito del flujo por nodos.
Inferencia	Ollama qwen2.5:3b	Modelo de lenguaje alojado localmente; tamaño moderado para demostración.
Embeddings	Ollama bge-m3	Representación vectorial de preguntas y documentos en español.
Vectores	ChromaDB	Persistencia de fragmentos, metadatos y búsqueda por distancia.
Datos estructurados	SQLite	Usuarios, tokens, conversaciones y registro de documentos.
Despliegue	Docker, Caddy, Nginx	Servicios autohospedados y proxy HTTPS en un VPS.

4.1 Principios de diseño

Separación de responsabilidades, configuración por variables de entorno, almacenamiento persistente, mínimos privilegios, manejo de errores y pruebas reproducibles. No se utilizan herramientas drag-and-drop; las decisiones del agente están escritas en Python.

5. Descripción del backend

5.1 Archivo principal y persistencia

backend/app/main.py: inicia FastAPI, crea tablas SQLite si no existen, valida solicitudes y autentica con tokens Bearer. Las tablas son usuarios, sesiones, conversaciones y documentos. La capa de autenticación almacena hash scrypt de contraseñas con salt aleatorio y SHA-256 de tokens de sesión; los tokens en claro solo se entregan al cliente al iniciar sesión o registrarse.

5.2 Rutas principales

Método	Ruta	Función
POST	/api/auth/register	Crear cuenta y sesión
POST	/api/auth/login	Iniciar sesión
GET	/api/auth/me	Perfil autenticado
POST	/api/auth/logout	Revocar sesión
POST	/api/chat	Generar respuesta y guardar historial
GET	/api/chat/history	Listar últimas 100 interacciones
DELETE	/api/chat/history	Vaciar el historial propio
GET	/api/documents	Listar documentos propios
POST	/api/documents	Subir y procesar archivo
DELETE	/api/documents/{id}	Eliminar documento y vectores
GET	/api/health	Salud (requiere token)

5.3 Flujo de pregunta

POST /api/chat valida tamaño de pregunta (2 a 2.000 caracteres), obtiene el identificador del usuario autenticado, invoca consultar_attelia y registra pregunta, respuesta, fuentes y hora. Si falla el servicio Ollama, devuelve 503 con mensaje controlado. Las consultas históricas están filtradas por usuario_id y limitadas a 100 registros.

5.4 Gestión documental

Se incorporaron GET, POST y DELETE /api/documents. La subida comprueba extensión, tamaño y contenido; extrae texto, lo divide e indexa en una colección exclusiva del usuario, y almacena un registro SQLite. Si falla la indexación, elimina el registro y el archivo y trata de limpiar vectores parciales. La eliminación requiere que el documento pertenezca al usuario antes de borrar sus vectores.

6. Descripción del frontend

6.1 Interfaz existente

El componente frontend/src/App.jsx gestiona estados React para autenticación, sesión, conversaciones, mensajes y errores. La experiencia incluye pantalla de acceso, barra lateral de historial, sugerencias de preguntas, compositor conversacional y visualización de fuentes. frontend/src/style.css conserva el estilo editorial crema, beige y acentos oscuros.

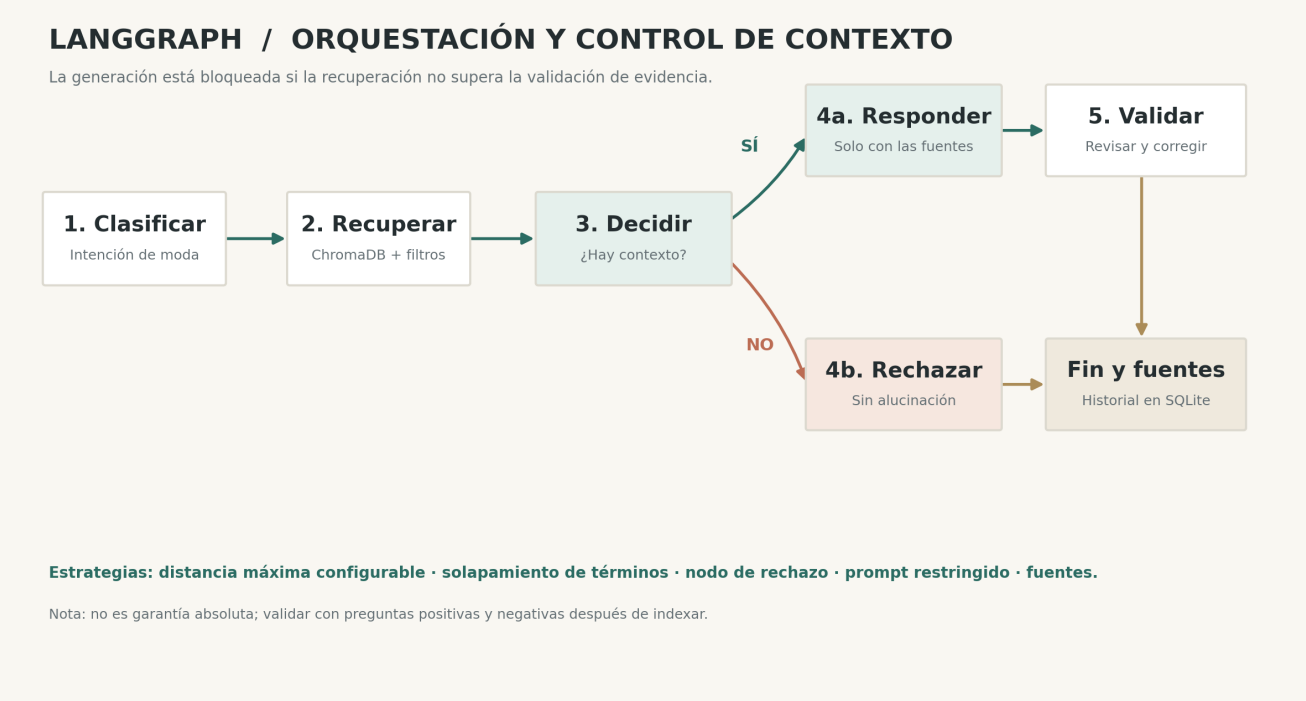
6.2 Gestión de archivos incorporada

El botón del clip, originalmente inactivo, abre un selector de archivos. Un FormData con campo «archivo» viaja a POST /api/documents sin establecer Content-Type manualmente, de manera que el navegador suministra el límite multipart. Se agregaron un panel «Mis documentos», confirmación de fragmentos indexados, listado privado, eliminación con confirmación y estados de error/carga.

6.3 Comunicación y sesión

La función request incorpora el encabezado Authorization Bearer, interpreta errores JSON y protege llamadas autenticadas. El token está en sessionStorage; esto conserva la sesión durante la pestaña, aunque no equivale a protección frente a XSS. La UI se muestra únicamente después de validar /api/auth/me.

7. Descripción del agente y flujo LangGraph



Elaboración propia a partir del código fuente de AtteliA.

7.1 Estado del grafo

EstadoAttelia (TypedDict) transporta pregunta, usuario_id, categoría, contexto, fuentes, indicador de suficiencia, respuesta, errores, necesidad de revisión y rechazo. StateGraph compila los nodos y sus transiciones; el diseño es código escrito explícitamente, no automatización visual.

7.2 Secuencia de nodos

- clasificar: determina si la consulta corresponde a calzado, prenda, accesorio, color, combinación, ocasión o general; usa heurísticas y el modelo si hace falta.
- recuperar: consulta la colección común y, si hay usuario, su colección privada. Aplica filtros de distancia y coincidencia léxica, organiza pasajes y registra fuentes.

- decidir_contexto: función de ruteo: si hay evidencia suficiente continúa a responder; en caso contrario va directamente al nodo rechazar.
- rechazar: devuelve un mensaje explícito predefinido, fuentes vacías y marca rechazada=True, sin invocar el generador.
- responder: construye la instrucción con solo los fragmentos recuperados y solicita respuesta concisa a qwen2.5:3b.
- validar/corregir: aplica reglas de presentación y revisión adicional; si detecta problemas, corrige utilizando de nuevo el contexto documental.

7.3 Condiciones de transición

```
START → clasificar → recuperar → (contexto suficiente?)
  NO → rechazar → END
  SÍ → responder → validar → (corregir?) → END
```

Una decisión relevante es impedir la vía de «conocimiento general» que existía en la primera versión. El flujo ahora prioriza la trazabilidad y acepta falsos rechazos a cambio de no generar sin evidencia.

8. Modelo local utilizado y base vectorial

8.1 Modelo principal y de embeddings

qwen2.5:3b se usa para clasificación auxiliar, generación y revisión. Es local mediante Ollama, no se realiza llamada a una API LLM comercial. Se eligió por su tamaño manejable para una demostración académica, aunque puede perder precisión ante instrucciones complejas. bge-m3 produce embeddings para el índice Chroma; hay que descargar ambos modelos antes de indexar o responder.

8.2 Organización del índice

La colección «attelia_conocimiento» almacena seis archivos docs/preparados/volumen_1.txt a volumen_6.txt. Cada usuario dispone de «attelia_usuario_{id}» para archivos cargados. Se usan fragmentos de aproximadamente 1.100 caracteres con 160 caracteres de solapamiento; cada fragmento conserva metadatos de fuente y, para documentos propios, ID de documento e ID de usuario.

8.3 Recuperación y límites

La búsqueda similarity_search_with_score obtiene fragmentos y distancia. Se retienen aquellos con distancia no superior a RAG_MAX_DISTANCE (valor inicial 1.35) y que comparten términos informativos con la consulta; se ordenan por distancia, se eliminan duplicados y se toman hasta cuatro. El umbral depende de embeddings, métrica e índice; se debe calibrar con pruebas sobre el corpus real, y no garantiza correspondencia factual por sí solo.

8.4 Reconstrucción

```
ollama pull qwen2.5:3b
ollama pull bge-m3
PYTHONPATH=backend python -m app.vectorstore.rag
```

Los vectores preexistentes del ZIP original no se incluyen en el paquete optimizado para reducir peso y evitar datos de ejecución; se reconstruyen a partir de los seis volúmenes. El índice y la base SQLite

persisten en el directorio data/ de la ejecución.

9. Proceso de carga de documentos

9.1 Fases operativas

- El navegador envía el archivo en FormData usando Authorization Bearer.
- FastAPI valida extensión entre PDF, DOCX, TXT, MD y CSV y un máximo configurable de 10 MB.
- El servidor lee el contenido y extrae texto con pypdf, python-docx o decodificación UTF-8.
- Se rechazan archivos sin suficiente texto y contenidos sobre el límite de caracteres.
- RecursiveCharacterTextSplitter genera fragmentos con 160 caracteres de solapamiento.
- bge-m3 calcula embeddings y Chroma los persiste en colección separada por usuario.
- La API confirma nombre e intensidad de indexación (cantidad de fragmentos).
- La consulta posterior puede recuperar el nuevo material con su fuente original.

9.2 Consistencia y eliminación

Cada documento tiene un identificador SQLite; ese ID se incorpora a la metadata vectorial para borrar únicamente los pasajes de ese archivo. Se intenta revertir el archivo, la fila y los vectores parciales ante errores. No existe transacción distribuida entre SQLite y Chroma: en fallos extremos puede requerirse intervención administrativa.

9.3 Formatos excluidos del alcance

JPG, JPEG, audio y video aparecen solo como formatos sugeridos en el enunciado, no como requisitos obligatorios. Esta implementación se centra en documentos de texto; los PDF compuestos solamente por imágenes requieren una etapa OCR que todavía no existe.

10. Estrategias implementadas para evitar alucinaciones

10.1 Defensas acumulativas

Mecanismo	Implementación	Efecto esperado
Filtro de distancia	RAG_MAX_DISTANCE y Chroma	Descarta fragmentos de baja proximidad vectorial.
Filtro léxico	Coincidencia de términos informativos	Reduce pasajes vagamente afines fuera del tema.
Decisión por grafo	decidir_contexto + nodo rechazar	Evita generación sin fragmentos aptos.
Prompt restringido	Solo afirmaciones sustentadas por contexto	Disminuye dependencia del conocimiento paramétrico.
Fuentes en respuesta	Nombres de documentos recuperados	Permite revisar trazabilidad.
Revisión posterior	Validación de respuesta y corrección	Detecta ciertos desvíos e inconsistencias de formato.

10.2 Respuesta negativa obligatoria

Mensaje: «No tengo suficiente información en mi base de conocimiento para responder esa pregunta. Puedes subir un documento relacionado». Es una ruta terminal directa del grafo y se guarda en historial como cualquier interacción.

10.3 Riesgos residuales

Aun con fragmentos relevantes, un modelo pequeño podría extrapolar más allá de la fuente. Los filtros pueden producir falsos negativos ante sinónimos o términos inflexionados. Para una versión productiva se recomienda evaluación humana periódica, pruebas de groundedness, reranking y métricas de precision/recall del recuperador.

11. Seguridad y autenticación

11.1 Controles presentes

- Registro y login con validación del formato del correo y contraseña mínima de 10 caracteres.
- Hash scrypt con salt aleatorio; comparación de hashes de tiempo constante.
- Sesión bearer generada con secrets.token_urlsafe, almacenada solamente como SHA-256 y expirada a los siete días; endpoint logout revoca el token.
- Todas las rutas de negocio requieren usuario_actual; registro e inicio de sesión son excepciones inevitables.
- Historial y metadatos documentales filtrados por usuario. Vectorstore separado por usuario.
- Extensiones, tamaño y extracción documental limitados; se usa un nombre de archivo interno aleatorio.
- Las rutas de documentación OpenAPI pública del backend están desactivadas; en nube Caddy ofrece terminación HTTPS.

11.2 Riesgos no resueltos

No se implementó recuperación de contraseña ni limitación distribuida de intentos de login; sessionStorage no mitiga por sí solo un XSS; no hay antivirus de archivos, OCR ni cifrado de los documentos en reposo. Las credenciales de producción nunca deben estar en el repositorio. Se recomienda backups, límites adicionales y seguimiento de vulnerabilidades de dependencias.

12. Despliegue

12.1 Arquitectura de despliegue preparada

docker-compose.yml define Ollama, un trabajo inicial de descarga de modelos, backend FastAPI, frontend Nginx y proxy Caddy. El backend no publica el puerto de Ollama. Caddy redirige /api/* a FastAPI y el resto a React sobre el mismo dominio; así se minimizan problemas de CORS y se habilita HTTPS con un nombre DNS válido.

12.2 Procedimiento previsto

- Crear repositorio público de GitHub con código, README, .env.example, capturas reales y diagramas.
- Contratar/provisionar VPS Linux con memoria adecuada y Docker Engine/Compose.
- Apuntar DNS A del dominio al servidor y abrir puertos TCP 80 y 443.
- Clonar repositorio y configurar .env a partir de .env.example (SITE_DOMAIN real).
- Ejecutar docker compose up -d --build y esperar la descarga de qwen2.5:3b y bge-m3.
- Indexar los seis volúmenes desde el contenedor backend con python -m app.vectorstore.rag.
- Verificar dominio frontend, login, API /api/health con token, RAG positivo, rechazo y subida.
- Tomar capturas reales y consignar URL frontend=https://dominio y backend=https://dominio/api.

12.3 Variables de entorno

Variable	Ejemplo / propósito
SITE_DOMAIN	Nombre de dominio para HTTPS Caddy; debe ser real.
OLLAMA_BASE_URL	http://ollama:11434 dentro del contenedor backend.
OLLAMA_MODEL	qwen2.5:3b, modelo de generación.
OLLAMA_EMBED_MODEL	bge-m3, modelo de embeddings.
RAG_MAX_DISTANCE	1.35 de partida; calibrar con ejemplos.
ATTELIA_DB	/app/data/attelia.db, persistencia de usuarios e historial.
CHROMA_DIR	/app/data/chroma, persistencia vectorial.
UPLOAD_DIR	/app/data/uploads, archivos propios.
FRONTEND_ORIGINS	Lista explícita de orígenes locales o autorizados.
VITE_API_URL	Vacío en despliegue de mismo dominio; localhost:8000 en desarrollo.

12.4 Estado real al entregar

Sin URL verificada a la fecha de generación de la memoria. El despliegue exige dominio, VPS y acceso a GitHub ajenos a este entorno. Se entregan configuraciones e instrucciones para completarlo; NO se declara una nube operativa hasta comprobarla.

13. Evidencias de funcionamiento

13.1 Auditoría del archivo de partida

La revisión del ZIP original encontró React + Vite en frontend, FastAPI + SQLite en backend, LangGraph con nodos de clasificación/recuperación/generación/validación, Ollama qwen2.5:3b y Chroma con seis volúmenes. Se encontró un botón de adjunto sin función y una ruta explícita que permitía responder con conocimiento general cuando no se recuperaban fuentes.

13.2 Cambios implementados sobre la base

- Nuevo servicio backend/app/services/documentos.py con extracción, fragmentación e indexado por usuario.
- Tres endpoints REST para cargar, listar y borrar documentos propios; manejo de errores y limpieza ante fallos.
- Nuevo filtro RAG y nodo condicional de rechazo sin generación.
- Pantalla «Mis documentos», botón de clip operativo, confirmación de indexado y fuentes existentes.
- Tests HTTP, tests específicos LangGraph, Dockerfiles, Compose, Caddy, README, guion, plan manual y diagramas.

13.3 Resultado de pruebas automáticas en el entorno de autoría

Resultado reproducible registrado: «3 passed, 3 skipped». Los tres tests HTTP pasados usaron FastAPI TestClient y verificaron autorización, sesiones, aislamiento de historial, flujo CRUD de documentos con servicio vectorial simulado y rechazo de extensión. Los tres módulos dependientes de langchain/langgraph/chroma quedaron omitidos aquí por ausencia de esas bibliotecas, por lo que no constituyen evidencia de ejecución real de Ollama.

13.4 Evidencias visuales pendientes y responsables

Evidencia	Cómo obtenerla	Estado
Registro/login	Captura del navegador con sesión de prueba.	PENDIENTE
RAG positivo	Pregunta de moda + respuesta + fuentes.	PENDIENTE
RAG negativo	Pregunta de sushi + mensaje explícito.	PENDIENTE
RAG incremental	Subida de TXT y consulta exclusiva del TXT.	PENDIENTE
Modelo local	Terminal `ollama list` mostrando ambos modelos.	PENDIENTE
Nube y GitHub	URL HTTPS real y enlace público del repositorio.	PENDIENTE

Para evidencias verificables, ejecutar la matriz en docs/pruebas-manuales.md y guardar capturas en docs/evidencias/. Deben eliminarse datos personales o credenciales de las capturas.

14. Problemas encontrados y conclusiones

14.1 Problemas y soluciones

Problema observado	Acción tomada / decisión
WinRAR fallaba al comprimir WSL por symlinks en node_modules/.bin	Empaquetar desde Ubuntu y excluir dependencias y datos pesados.
Frontend mostraba «Carga de documentos próximamente»	Implementar selector, FormData, lista y eliminación.
El grafo permitía usar conocimiento general sin evidencia	Eliminar fallback y añadir decisión hacia nodo rechazar.
Base Chroma grande y dataset Polyvore incluido en ZIP	Mantener corpus TXT y script de reindexación reproducible; excluir dataset pesado.
Credenciales y equipo de despliegue desconocidos	Entregar artefactos Docker, checklist y espacios para URL reales.
Dependencias LangChain no instaladas en el entorno de autoría	Ejecutar solo tests API disponibles y dejar constancia de pruebas omitidas.

14.2 Conclusiones

AtteliA muestra un diseño técnicamente coherente para una aplicación web documental: interfaz, autenticación, agente dirigido por grafo, inferencia local, índice vectorial y una ruta explícita de rechazo. La integración de archivos de usuario permite ampliar la base sin reentrenar el LLM y preserva una separación lógica por cuenta. El resultado es más verificable que una generación de texto sin fuentes, aunque no garantiza respuestas perfectas.

La culminación académica requiere además comprobar en Ubuntu el funcionamiento de Ollama con los dos modelos, calibrar la recuperación, ejecutar todos los tests, demostrar documentalmente una ingesta nueva y completar la publicación pública. El criterio del profesor indica que un despliegue en la nube y un repositorio documentado son mínimos obligatorios; por tanto, **no basta la sola preparación de Docker para declarar el proyecto totalmente aprobado**.

Anexo A · Matriz de trazabilidad de la rúbrica

Requisito del profesor	Evidencia de implementación	Qué falta verificar
Backend/frontend	backend/app/main.py; frontend/src/App.jsx	Ejecutar ambos localmente y captar pantallas.
Autenticación	Sesiones Bearer y autorización FastAPI	Confirmar prueba visual y seguridad en nube.
LangChain/LangGraph	backend/app/agents/graph.py	Ejecutar prueba de grafo con dependencias.
Modelo local	ChatOllama qwen2.5:3b	ollama list y consultas reales.
Vector y carga	rag.py; documentos.py; endpoints	Reindexar, subir y volver a consultar.
RAG fundamentado	Filtros, prompts, fuentes	Auditar respuesta vs pasajes recuperados.
Rechazo	decidir_contexto → rechazar	Capturar pregunta no cubierta.
Nube / GitHub	Compose, Dockerfiles, README	Subir repositorio y publicar URLs.
Documento / slides	documento-tecnico.pdf y presentacion.pdf	Completar datos personales y evidencias.

Anexo B · Reproducibilidad

```
cd backend && python3 -m venv .venv
source .venv/bin/activate && pip install -r requirements.txt
cd .. && ollama pull qwen2.5:3b && ollama pull bge-m3
PYTHONPATH=backend python -m app.vectorstore.rag
PYTHONPATH=backend uvicorn app.main:app --reload
cd frontend && npm ci && npm run dev
./scripts/verificar_backend.sh
```

En Docker con DNS/TLS: docker compose up -d --build; docker compose logs -f ollama-init; docker compose exec backend python -m app.vectorstore.rag. Las instrucciones detalladas están en README.md.

Anexo C · Secuencia recomendada de demostración

1) login; 2) consulta positiva y fuente; 3) rechazo por consulta sin documentos; 4) subir TXT con hecho único; 5) pregunta de ese TXT; 6) mostrar código de grafo; 7) mostrar HTTPS, GitHub y terminal Ollama. Ver docs/guion-sustentacion.md para el guion cronometrado.

Anexo D · Fuentes y material de referencia

Enunciado académico: «Trabajo Agente — Desarrollo de un agente inteligente con LangChain, LangGraph y RAG», diez páginas. Corpus de trabajo: seis volúmenes de moda provistos en docs/preparados/, con inventario en docs/fuentes.md. Material Polyvore experimental en el repositorio original; no es requisito ni parte del flujo principal de esta entrega.

Declaración de alcance de evidencias

Los diagramas presentados describen el diseño implementado en código. Son representaciones técnicas, NO capturas de funcionamiento. Los escenarios HTTP probados usaron simulación de la capa de indexado; no se equiparan con una validación del despliegue real o del modelo Ollama en Ubuntu. Se han reservado explícitamente los apartados que requieren intervención del equipo.

ANEXO E · EVIDENCIAS REALES

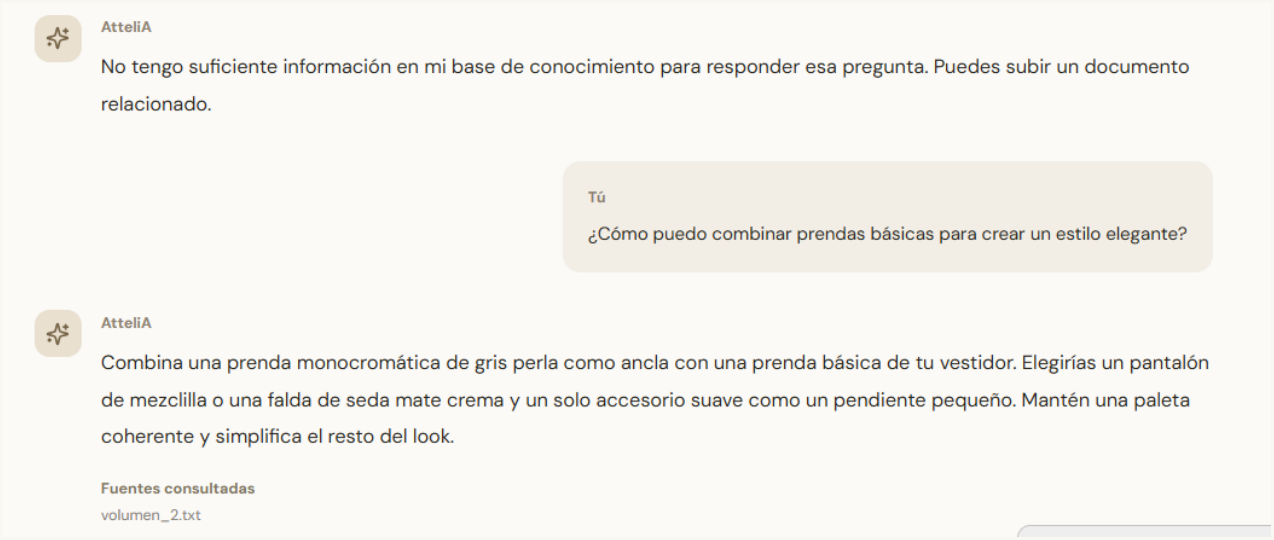
Actualización tras las pruebas funcionales efectuadas el 18 de octubre de 2026 (hora Colombia).

E1 · Respuesta basada en documentos



La pregunta sobre el armario cápsula devolvió una respuesta con referencias a volumen_1.txt y volumen_4.txt. En la misma captura se observa una consulta sobre fútbol que AtteliA rechaza explícitamente.

E2 · Recomendación de vestuario con fuente

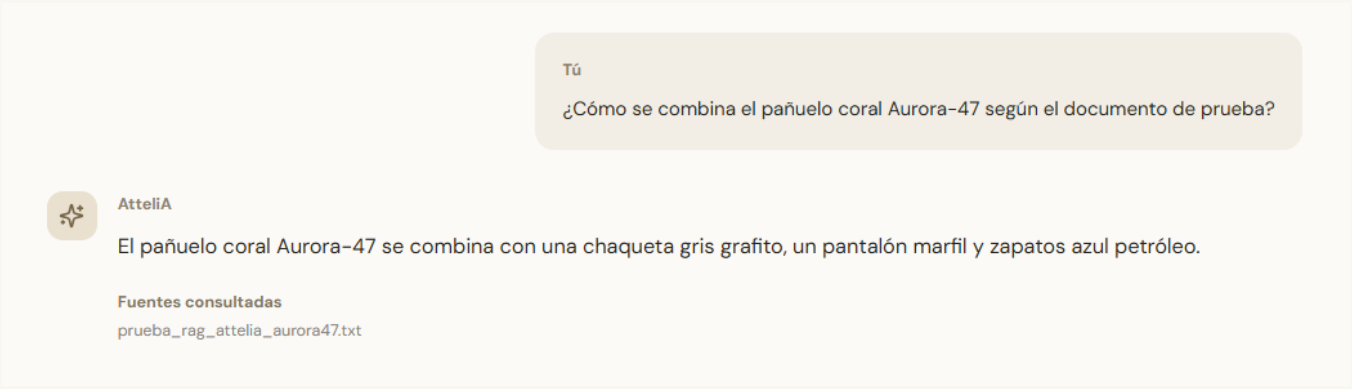


La consulta sobre prendas básicas y estilo elegante muestra una respuesta con la fuente volumen_2.txt.

ANEXO F · INGESTA INCREMENTAL

Prueba de un documento TXT nuevo y corrección del comportamiento de respuesta.

E3 · Consulta tras cargar el TXT Aurora-47



El agente recuperó prueba_rag_attelia_aurora47.txt y respondió con la combinación literal del documento: chaqueta gris grafito, pantalón marfil y zapatos azul petróleo.

Incidencia detectada y resuelta

La primera consulta tras cargar el archivo mostró su fuente, pero mezcló recomendaciones generales y produjo una combinación incorrecta. Se corrigió la recuperación para priorizar identificadores específicos y extraer respuestas documentales cuando corresponde. El resultado mostrado arriba se verificó después de aplicar el parche.

Alcance de la evidencia

Estas capturas verifican el comportamiento observado en el equipo local. No constituyen evidencia de un despliegue en la nube, de pruebas de carga ni de ausencia absoluta de alucinaciones. Las direcciones públicas continúan pendientes de verificación.